MMAUG

AI & DEVOPS BOOTCAMP

PROJECT 02

School Login Portal

Step-by-step Build Guide

Project	MMAUG High School Secure Login Portal
Framework	Python FastAPI
Identity	Microsoft Entra ID test tenant
RFP Reference	MMAUG-AIDB-RFP-002
Prepared	3 November 2026

Plan → Build → Secure → Test → Deploy → Demonstrate

This guide turns **the accepted proposal** into a working MVP. We will complete and verify one milestone at a time. The project uses a separate test Entra tenant with cloud-only synthetic accounts, an available Azure subscription, and a guide-and-run workflow: the user runs commands with assistance.

How we will work

1. Agree the assumptions that affect identity, student data, and recovery.
2. Build the smallest working FastAPI app locally.
3. Add PostgreSQL, authentication, roles, and ownership checks in short slices.
4. Run the relevant tests and a manual demonstration after each slice.
5. Deploy only after security, backup, monitoring, and acceptance evidence exist.

I will prepare each command, explain its result, and review the output you share. You will run commands and complete account steps on your computer; never paste passwords, client secrets, tokens, or recovery codes here. We will record what actually passed before moving on.

Milestones

Step	Build	Done when
0. Confirm decisions	Test tenant and cloud-only accounts confirmed; check app-registration/admin rights, MFA and reset ownership	Decisions recorded; no unclear rule is silently implemented
1. Prepare tools	Python 3.13, virtual environment, Git; Docker Desktop before database work	Real Python interpreter and Git commands work; local venv created
2. Start FastAPI	Project package, config, `/health`, basic page, smoke test	App runs locally; `/health` and `/docs` respond; test passes
3. Add data	PostgreSQL, SQLAlchemy models, Alembic migrations, synthetic account mappings	Database can be recreated from migrations; test identities map by Entra tenant/object ID
4. Secure sign-in	Single-tenant Entra OIDC, authorization-code flow, opaque app sessions, sign-in/out, CSRF	Valid Entra login works; unknown, inactive, expired, and revoked sessions fail
5. Enforce access	Student/teacher/admin dashboards and protected APIs; record ownership	Cross-role and cross-student requests are denied server-side
6. Recover and administer	Administrator-assisted Entra reset, admin MFA, portal role changes, and authorized Graph account disable/enable	A synthetic user's whole Entra account is disabled/enabled only by an authorized operator; portal sessions close; reset is demonstrated by the approved helpdesk route
7. Observe and test	Audit view, alerts, integration/browser/security/load tests	Required eight-user demo and 1,500-account scale report pass
8. Deliver	Container, CI/CD, infrastructure as code, UAT/production separation, backup and rollback	Tested image deploys to UAT; release gates and handover evidence pass

Step 0: decisions to record before identity code

- **Identity:** Use the separate test Microsoft Entra tenant for authentication during the MVP. Register the portal as a **single-tenant web application**. FastAPI will accept only validated sign-ins from the selected tenant, then map the immutable Entra tenant ID and user object ID to a pre-provisioned portal account. No local password database or self-registration is planned.
- **Recovery:** SSPR is **disabled** and the test accounts are **cloud-only**. Use an approved administrator/helpdesk password reset for the demo. A suitably authorized Entra administrator performs the reset in the Entra admin center; the portal can show help instructions or route the request but will not collect Entra passwords or temporary passwords.
- **Data access:** The MVP will let students view only their own minimal profile. Teachers will not receive a student roster until an explicit use case and access rule are approved.
- **Roles and deactivation:** Portal roles remain in PostgreSQL. You specified that deactivation must disable the user's **whole Entra account**, affecting other Microsoft-connected services. The eventual portal action will use a delegated Microsoft Graph call made on behalf of a signed-in Entra administrator who also holds the required directory role. Graph identifies the target by immutable object ID, and the app will block self-targeting, privileged/break-glass accounts, and targets outside the approved test scope. A local portal flag alone does not satisfy this requirement. The portal must revoke its own sessions immediately; the directory sign-in sessions should also be revoked with an approved Graph permission. If Graph fails, show the directory action as failed and keep an audit record.
- **MFA:** Administrators must use MFA through an approved Entra policy. The teacher requirement remains a school decision. Verify the tenant's licensing and policy before relying on a particular Conditional Access feature.
- **Hosting:** A separate test Entra tenant and an Azure subscription are available, but local development comes first. Confirm that the subscription belongs to the intended tenant and check region, domain, resource-creation permissions, and budget before provisioning.

Record any changed assumption in the **decision log** before implementing it. Use synthetic names and email addresses throughout development and UAT. The existing proposal describes local passwords and portal-only deactivation as baselines; the confirmed Entra design and tenant-wide deactivation requirement above supersede those baselines for implementation. Directory writes remain pending until an authorized tenant administrator approves and configures the exact permissions.

Your read-only Entra check

1. Open the Microsoft Entra admin center and switch to the separate test tenant. Note its directory name. Keep passwords and sign-in codes private.
2. Open **Entra ID > App registrations**. Check whether **New registration** is available to you. Do not create the app yet; we will set the exact callback URL together when the local app is ready.
3. In **Entra ID > Users**, check that the intended synthetic test accounts are present or can be created in this cloud-only tenant. Do not modify any account yet.
4. Confirm who can support app registration, Microsoft Graph permission consent, and a test password reset. The operator who disables non-admin users must hold an appropriate Entra role; the app's portal-admin role alone is insufficient.
5. Ask the tenant administrator whether the intended portal administrators already have MFA enforced for this application or whether a policy must be configured. Do not change Conditional Access during this check.

6. In the Azure portal, check that you can see the intended subscription. Report only whether you can create resources there or need an administrator to assign a resource group.

Share the six results as short status lines. You may share tenant/client IDs later if needed; never share a client secret, access token, password, MFA code, or recovery code. Do not test deactivation or password reset on a real student/staff account.

Step 1: prepare this Windows computer

The current workspace has Git, but `python` resolves to a Windows Store alias; `py`, `pip`, and Docker Desktop were not found in the shell check on 3 November 2026. Install the official **Python 3.13 x64 full installer** for the current user, with `pip` and the Python launcher. Then reopen PowerShell and run:

```
py -3.13 --version
py -3.13 -m pip --version
git --version
```

The first two commands must show the installed Python and pip versions. If `py` is still missing, use the installed `python.exe` path and check Windows **Manage app execution aliases**. Do not use the WindowsApps placeholder as proof that Python is installed. We can use the virtual environment's `python.exe` directly later, avoiding changes to PowerShell's script execution policy.

Docker Desktop is needed for the planned local PostgreSQL setup, but it can wait until Step 3. Its Windows setup may require WSL 2 and a restart. We will check those prerequisites when we reach the database milestone.

Evidence we will keep

- A requirement-to-test checklist and decision log.
- Source code with dependency versions, migrations, and synthetic seed data.
- CI results for formatting, unit, integration, browser, dependency, and secret checks.
- A role-permission matrix and tests for direct API access and another student's profile.
- An Entra app-registration record with exact redirect URIs and a test of tenant/object-ID mapping. Keep credentials out of this record.
- A delegated-permission review and audit trail for a synthetic account disable/enable and session-revocation test. No broad Graph application permission is assumed.
- A record of UAT deployment, backup restore, rollback rehearsal, and the eight-user demonstration.
- A load test showing 1,500 registered accounts and the agreed concurrent-user assumption.

Reference documentation

- Python on Windows
- FastAPI first steps
- FastAPI container deployment

- Docker Desktop on Windows
- Register a single-tenant app in Microsoft Entra ID
- Validate Entra claims using tenant and object IDs
- Graph permissions for updating `accountEnabled`
- Revoke a user's Entra sign-in sessions
- Reset a user's password in Entra